CO4-AT1 – Question 39

Name: Talari Vishnuvardhan

Register No: 192572091

Subject: Design and Analysis of Algorithms

Faculty: Dr. F. Mary Harin Fernandez

Q39. Decode Ways Problem

1. Problem Statement

A string of digits represents encoded letters where the digits from 1 to 26 correspond to the letters A to Z. The objective is to determine the total number of valid ways in which the given digit string can be decoded.

The problem is solved using Dynamic Programming, where previously calculated results are stored and reused to avoid repeated calculations.

For example, the input 226 can be decoded in three different ways:

- 2 2 6
- 22 6
- 2 26

Therefore, the expected answer is 3.

2. Objective

The main objectives are:

- To understand the Decode Ways problem.
- To implement the solution using Dynamic Programming.
- To calculate the number of possible valid decodings.
- To handle valid and invalid digit combinations.
- To analyze the time and space complexity of the solution.

3. Input

s = 226

4. Output

Number of decoding ways = 3

5. Approach Used

The problem is solved using Dynamic Programming.

A DP table is used to store the number of ways to decode each prefix of the input string.

At every position, two possibilities are considered:

1. The current digit is decoded as a single letter.
2. The current two digits are decoded together if they form a number between 10 and 26.

The results of smaller subproblems are used to calculate the result for larger subproblems.

6. Dynamic Programming Concept

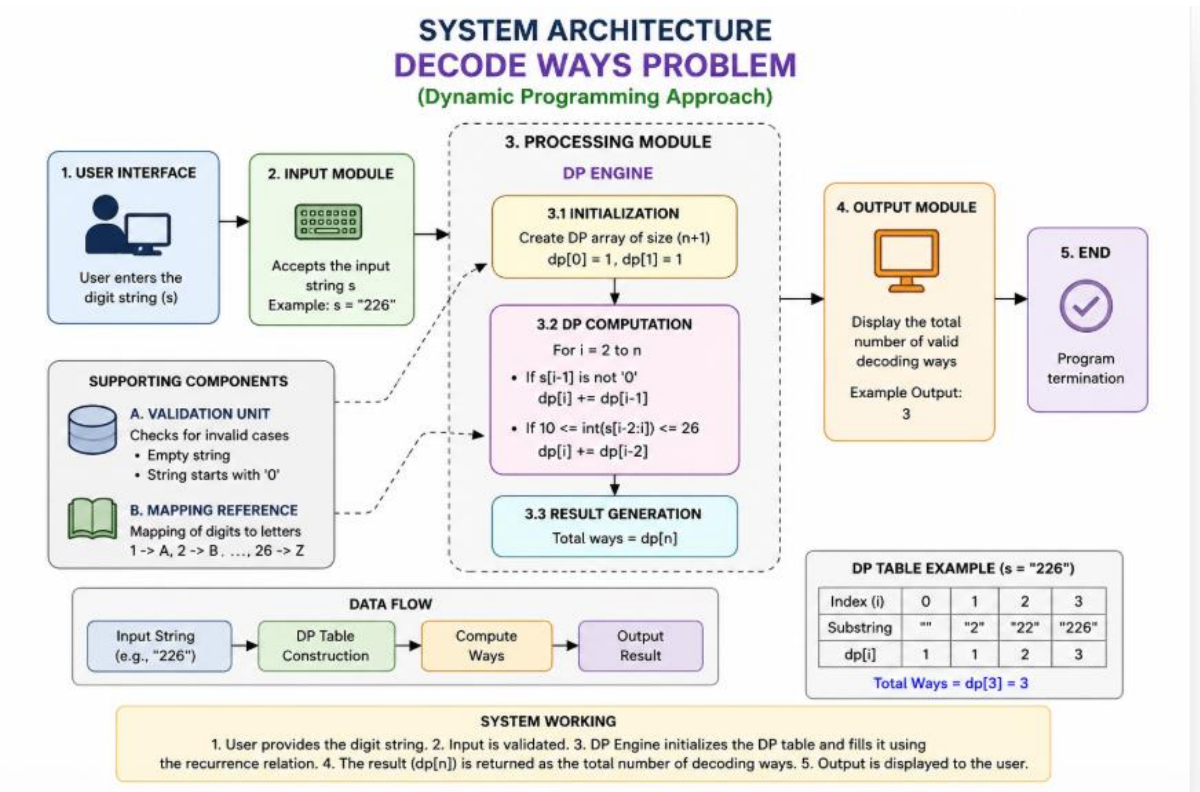
Let dp[i] represent the number of ways to decode the first i characters of the string.

For every position:

- If the current digit is between 1 and 9, it can be decoded individually.
- If the last two digits form a number between 10 and 26, they can be decoded together.

The final DP value represents the total number of valid decoding combinations.

System architecture



7. Step-by-Step Analysis

Given:

226

The possible divisions are:

2 | 2 | 6

22 | 6

2 | 26

Corresponding letters are:

2 → B

6 → F

22 → V

26 → Z

Therefore:

2 | 2 | 6 → BBF

22 | 6 → VF

2 | 26 → BZ

Hence, there are 3 valid ways.

8. DP Table

Position	Value	Number of Ways
0	Empty	1
1	2	1
2	22	2
3	226	3

Final result:

dp[3] = 3

9. Source Code

```python
def decode_ways(s):
    n = len(s)
    if n == 0 or s[0] == '0':
        return 0
    dp = [0] * (n + 1)
    dp[0] = 1
    dp[1] = 1
    for i in range(2, n + 1):
        if s[i - 1] != '0':
            dp[i] += dp[i - 1]
        two_digit = int(s[i - 2:i])
        if 10 <= two_digit <= 26:
            dp[i] += dp[i - 2]
    return dp[n]
s = input("Enter the digit string: ")
result = decode_ways(s)
print("Number of decoding ways =", result)
```

10. Execution / Output

```
Enter the digit string: 226
Number of decoding ways = 3
```

Expected output:

Enter the digit string: 226

Number of decoding ways = 3

11. Handling Invalid Input

The solution also handles invalid cases.

A string beginning with 0 is invalid because there is no letter corresponding to 0.

For example:

Input: 0

Output: 0

Similarly, invalid two-digit combinations such as 27, 30, or 45 cannot be treated as a single letter.

12. Recurrence Relation

The Dynamic Programming recurrence is:

If the current digit is valid:

dp[i] = dp[i] + dp[i-1]

If the last two digits form a number from 10 to 26:

dp[i] = dp[i] + dp[i-2]

Thus, the number of valid decoding possibilities is calculated efficiently.

13. Time Complexity

The input string is processed only once.

Time Complexity: O(n)

where n is the length of the digit string.

14. Space Complexity

A DP table of size n + 1 is maintained.

Space Complexity: O(n)

15. Advantages

- Uses an efficient Dynamic Programming approach.
- Avoids repeated calculations.
- Works efficiently for large input strings.
- Handles single-digit and two-digit decoding.
- Can identify invalid decoding combinations.
- Easy to understand and implement.

16. Result

The Decode Ways problem was successfully solved using the Dynamic Programming technique.

For the input:

226

the program produces:

Number of decoding ways = 3

The three valid decoding combinations are:

2 2 6

22 6

2 26

Therefore, the final result is 3 valid decoding ways.

17. Conclusion

The Decode Ways problem demonstrates the effective use of Dynamic Programming for solving problems involving multiple possible combinations. By storing the solutions of smaller subproblems and reusing them, the total number of valid decoding ways can be calculated efficiently. The solution has a time complexity of O(n) and a space complexity of O(n).